

# Integration Layer

App1 <-> PostgreSQL <-> App2, with a data warehouse layer

Version 1.0 | 28 September 2026

|          |                                                                              |
|----------|------------------------------------------------------------------------------|
| Document | Architecture, component analysis and database schema                         |
| System   | Integration Layer between App1 and App2 over PostgreSQL, with data warehouse |
| Stack    | Python 3.12, FastAPI, psycopg 3, PostgreSQL 16, Docker Compose               |
| Status   | Phases 1 to 4 built and tested; Docker run and hardening pending             |
| Audience | Engineers, architects and technical stakeholders                             |

# Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>1. Executive summary</b>                                | <b>4</b>  |
| 1.1 Confirmed decisions . . . . .                          | 4         |
| 1.2 Key properties . . . . .                               | 4         |
| <b>2. Architecture overview</b>                            | <b>5</b>  |
| 2.1 Design principles . . . . .                            | 5         |
| 2.2 End-to-end flow . . . . .                              | 6         |
| 2.3 Request lifecycle . . . . .                            | 7         |
| <b>3. Component analysis</b>                               | <b>7</b>  |
| 3.1 PostgreSQL 16 (postgres) . . . . .                     | 7         |
| 3.2 Migration runner (migrate) . . . . .                   | 8         |
| 3.3 Integration Layer API (il-api) . . . . .               | 8         |
| 3.4 Callback dispatcher (inside il-api) . . . . .          | 8         |
| 3.5 Scheduler (il-scheduler) . . . . .                     | 8         |
| 3.6 App2 (app2) . . . . .                                  | 9         |
| 3.7 App1 mock and client library (app1-mock) . . . . .     | 9         |
| <b>4. Integration Layer API</b>                            | <b>9</b>  |
| 4.1 Endpoints . . . . .                                    | 9         |
| 4.2 Submit request body . . . . .                          | 10        |
| 4.3 Error format and codes . . . . .                       | 10        |
| 4.4 Callback payload . . . . .                             | 10        |
| <b>5. Database schema</b>                                  | <b>11</b> |
| 5.1 Requirement traceability: the metadata table . . . . . | 11        |
| 5.2 integration.request_metadata . . . . .                 | 12        |
| 5.3 integration.workflow_registry . . . . .                | 13        |
| 5.4 integration.api_call_log . . . . .                     | 13        |
| 5.5 integration.request_event_log . . . . .                | 14        |
| 5.6 integration.idempotency_keys . . . . .                 | 14        |
| 5.7 Functions and procedures . . . . .                     | 14        |
| 5.8 Indexes . . . . .                                      | 15        |
| 5.9 Roles and privileges . . . . .                         | 16        |
| <b>6. Data warehouse layer</b>                             | <b>17</b> |
| 6.1 Load process . . . . .                                 | 17        |
| 6.2 fact_request . . . . .                                 | 17        |
| 6.3 Dimensions and control tables . . . . .                | 18        |
| 6.4 Reporting view . . . . .                               | 18        |

**7. Security, reliability and operations** **19**

7.1 Security controls . . . . . 19

7.2 Reliability mechanisms . . . . . 19

7.3 Configuration reference . . . . . 19

**8. Deployment** **20**

8.1 Adding a new workflow . . . . . 20

**9. Verification** **20**

**10. Status and roadmap** **21**

10.1 Next steps . . . . . 21

# 1. Executive summary

App1 and App2 never talk to each other directly. App1 sends JSON requests to an **Integration Layer** (a FastAPI service). The Integration Layer validates each request against a registered JSON Schema and stores it, with full metadata, in **PostgreSQL**. **App2** (also FastAPI) picks pending requests up from PostgreSQL, processes them, and writes the output JSON and the final status back. The Integration Layer then returns the result to App1, either when App1 polls or by pushing a signed callback. A **data warehouse** schema in the same database is loaded incrementally for reporting.

PostgreSQL plays three roles at once: the **message queue** between the applications, the **system of record and audit trail** for every request, and the **source for analytics**. At the expected volume of 500 to 1,000 requests per day, one PostgreSQL instance has headroom of several orders of magnitude, and there is no separate message broker to run.

## 1.1 Confirmed decisions

| Topic           | Decision                                                 | Effect on the design                                                                                                   |
|-----------------|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Result delivery | Polling and callback                                     | GET /v1/requests/{id} always works; if a callback_url is given, the result is also POSTed, signed, retried and tracked |
| Technology      | Python 3.12 + FastAPI for the Integration Layer and App2 | Shared libraries: psycopg3 (async pool), pydantic-settings, jsonschema, httpx                                          |
| Volume          | 500 to 1,000 requests/day                                | One replica per service; no PgBouncer or broker; monthly partitions                                                    |
| Hosting         | Docker containers                                        | Docker Compose stack; a Python scheduler container replaces pg_cron                                                    |
| Validation      | JSON format validation required                          | workflow_registry holds input and output JSON Schemas; the IL validates input, App2 validates output                   |

## 1.2 Key properties

- **Every request is fully recorded:** input JSON, output JSON, API URL and method, server details, timestamps for each stage, workflow completion status from App2, and a success or failure flag.
- **Nothing is processed twice by accident:** workers claim rows with FOR UPDATE SKIP LOCKED; only the worker holding the lease can report a result; idempotency keys stop duplicate submits.
- **Nothing gets stuck:** a lease with heartbeats, a reaper job and retries with exponential backoff recover from App2 crashes and temporary errors.
- **Least privilege:** App1 has no database access at all. App2 can only execute four queue functions. Reporting users can read only the warehouse schema.
- **Verified:** 29 automated tests and a 21-check end-to-end run pass against PostgreSQL 16.

## 2. Architecture overview

The diagram shows the runtime components and how they communicate. Solid arrows are requests or SQL calls; dashed arrows are asynchronous signals (PostgreSQL NOTIFY) and callbacks.

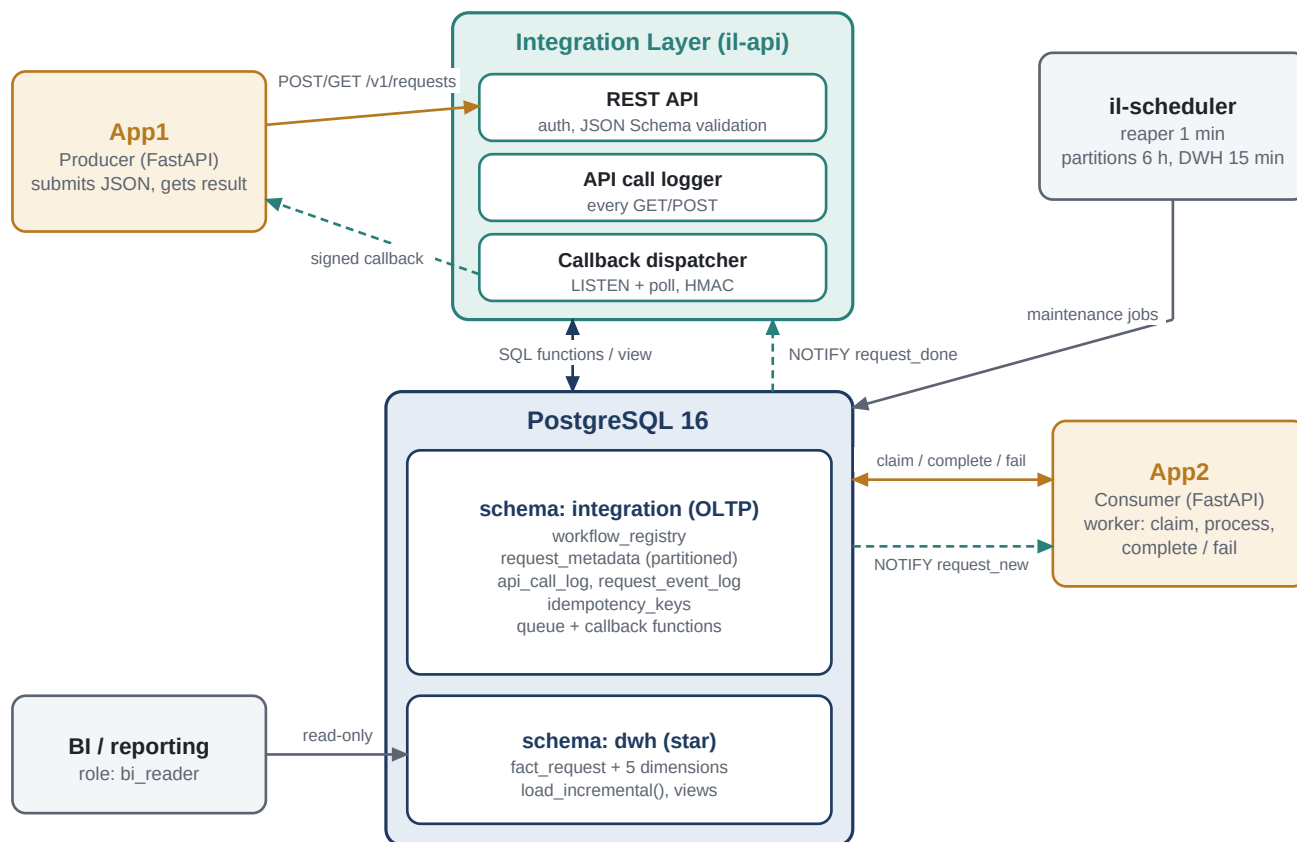


Figure 1. System architecture

### 2.1 Design principles

| Principle                                     | How it is applied                                                                                                                            |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Decoupling                                    | App1 and App2 share only a contract (workflow name plus JSON Schemas). Either can be down without losing requests; work waits in PostgreSQL. |
| Database as the single source of truth        | Queue state, payloads, audit history and API logs live in one transactional store, so status can never disagree with data.                   |
| State changes only through SQL functions      | submit, claim, heartbeat, complete, fail, reap, cancel and replay enforce the lifecycle in one place.                                        |
| Push with a pull fallback                     | LISTEN/NOTIFY gives millisecond reaction; periodic polling covers missed notifications and restarts.                                         |
| Validate at the edges                         | Input is validated before it is stored; output is validated before it can reach App1.                                                        |
| Separate operational and analytical workloads | Reporting reads the dwh star schema, not the queue tables.                                                                                   |

## 2.2 End-to-end flow

The sequence below follows one request from submission to delivery of the result.

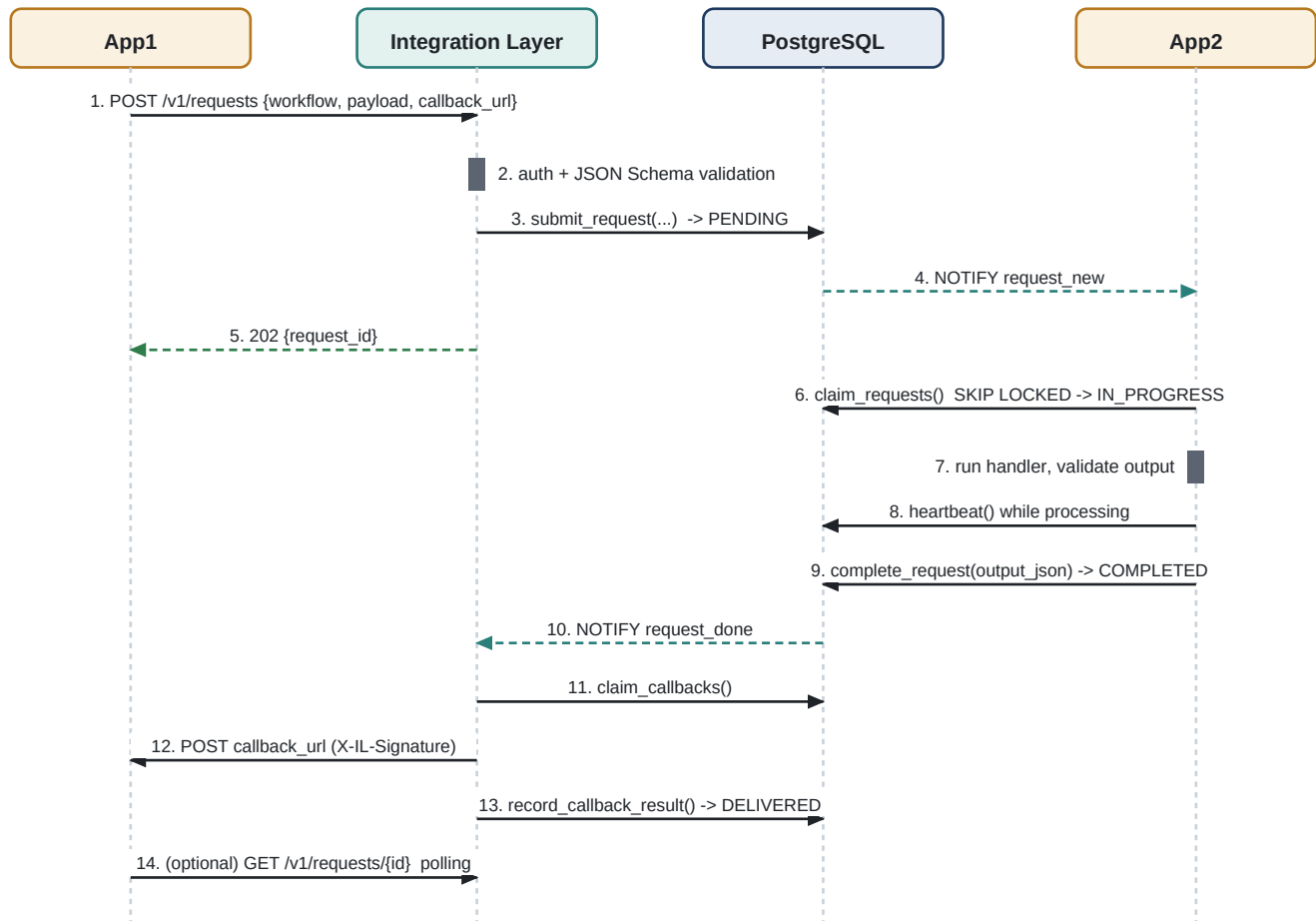


Figure 2. Request sequence (callback mode, with optional polling)

| Step  | What happens                                                                                                                                                                                                                                                                                                     |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-2   | App1 calls POST /v1/requests with its API key. The IL authenticates the caller, derives source_app from the key, and validates the payload against the workflow's input_schema. Invalid payloads are rejected with HTTP 422 and a list of errors by JSON path; nothing is stored.                                |
| 3-5   | submit_request() inserts the row with status PENDING and all metadata (URL, method, masked headers, client IP, IL host and IP, schema version). A trigger writes the audit event and sends NOTIFY request_new. App1 receives 202 with the request_id (200 with the original id for a duplicate idempotency_key). |
| 6-9   | An App2 worker wakes on the notification, claims the row (IN_PROGRESS, lease 120 s), runs the workflow handler, sends heartbeats for long jobs, validates the output against output_schema and calls complete_request() or fail_request().                                                                       |
| 10-13 | On a terminal status a trigger queues the callback and sends NOTIFY request_done. The IL dispatcher claims the callback, POSTs the signed result to App1 and records DELIVERED, or schedules a retry.                                                                                                            |
| 14    | Independently, App1 can poll GET /v1/requests/{id} at any time.                                                                                                                                                                                                                                                  |

## 2.3 Request lifecycle

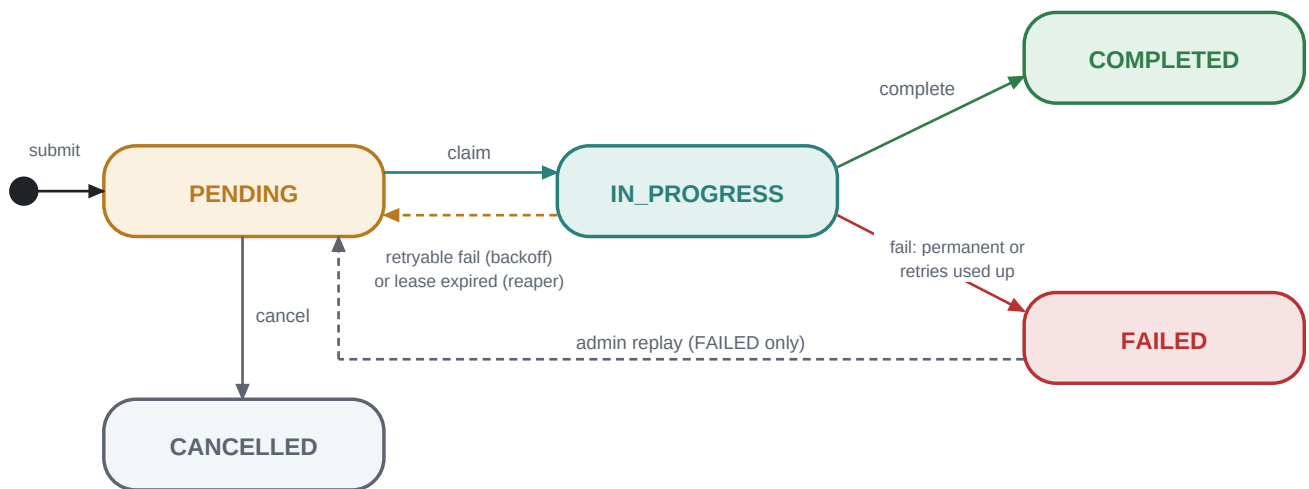


Figure 3. Request state machine

| Status      | Meaning                                                                            | success_flag |
|-------------|------------------------------------------------------------------------------------|--------------|
| PENDING     | Stored and waiting for App2: new, scheduled for retry, or recovered by the reaper  | NULL         |
| IN_PROGRESS | Claimed by an App2 worker holding an active lease                                  | NULL         |
| COMPLETED   | App2 finished the workflow; output_json and workflow_status are stored             | TRUE         |
| FAILED      | Permanent error, invalid output, or retries used up; acts as the dead-letter state | FALSE        |
| CANCELLED   | Cancelled by App1 while still PENDING                                              | FALSE        |

A check constraint ties success\_flag to status, so the two can never disagree. Retry backoff is  $30 \text{ s} \times 2^n$ , capped at 1 hour; max\_retries comes from the workflow registry (default 3).

## 3. Component analysis

Each component below lists its responsibility, internals, configuration, failure behaviour and scaling. Source paths are relative to the project root.

### 3.1 PostgreSQL 16 (postgres)

| Aspect            | Detail                                                                                                                                                                                                            |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Responsibility    | Queue, system of record, audit trail and data warehouse in one database named <code>integration</code> .                                                                                                          |
| Schemas           | <b>integration</b> : operational tables and functions. <b>dwh</b> : star schema, load procedure and reporting views.                                                                                              |
| Key mechanisms    | FOR UPDATE SKIP LOCKED claims; LISTEN/NOTIFY channels request_new and request_done; triggers for audit, updated_at and callback scheduling; monthly range partitions on request_date; SECURITY DEFINER functions. |
| Image / storage   | postgres:16, named volume pgdata, port bound to 127.0.0.1:5432 only.                                                                                                                                              |
| Failure behaviour | Services reconnect automatically. No request is lost because it is committed before App1 gets 202.                                                                                                                |
| Scaling           | Single instance is ample for the stated volume. Later: read replica for reporting, PgBouncer for many connections.                                                                                                |

### 3.2 Migration runner (migrate)

| Aspect         | Detail                                                                                                                                                                         |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source         | integration_layer/migrate.py; runs once per deployment as a one-shot container.                                                                                                |
| Responsibility | Applies db/*.sql in filename order inside transactions, records each file with a SHA-256 checksum in public.schema_migrations, and creates or updates the service login users. |
| Safety         | Takes an advisory lock so two runs cannot overlap. Refuses to continue if an applied file was changed afterwards: schema changes must go in a new numbered file.               |
| Login users    | il_api (il_service), app2 (app2_worker), il_scheduler_user (il_scheduler + dwh_etl), bi_reader (dwh_reader). Passwords come from MIGRATE_* variables.                          |

### 3.3 Integration Layer API (il-api)

| Aspect            | Detail                                                                                                                                                                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source            | integration_layer/ : main.py, routes/, validation.py, api_log.py, callbacks.py, auth.py                                                                                                                                           |
| Responsibility    | The only entry point for App1: accept, validate, store, report status, cancel; admin management of workflows and replays; callback delivery.                                                                                      |
| Authentication    | X-API-Key header mapped to a source_app (IL_API_KEYS). Keys are compared in constant time. Apps can only see and cancel their own requests. A separate admin key protects /v1/admin/*.                                            |
| Validation        | Pydantic validates the request envelope (unknown fields rejected). The payload is validated with JSON Schema draft 2020-12 including format checks; the schema catalog is cached for 30 s. Payloads over 1 MB are rejected (413). |
| Metadata capture  | Full URL, method, masked headers (Authorization, X-API-Key, Cookie), client IP (X-Forwarded-For aware and validated), IL hostname, IP and instance id.                                                                            |
| API call logging  | Middleware records every inbound call (method, URL, route template, status, latency, error body) in api_call_log. Writes are asynchronous so logging never delays or breaks a response.                                           |
| Failure behaviour | Stateless; a restart loses nothing. If the database is down, /health/ready returns 503.                                                                                                                                           |
| Scaling           | Horizontal: any number of replicas behind a load balancer.                                                                                                                                                                        |

### 3.4 Callback dispatcher (inside il-api)

| Aspect   | Detail                                                                                                                                                                                                                                                                                            |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source   | integration_layer/callbacks.py                                                                                                                                                                                                                                                                    |
| Trigger  | Wakes on NOTIFY request_done and also polls every 5 s, which covers missed notifications and retries that become due.                                                                                                                                                                             |
| Delivery | claim_callbacks() takes due callbacks with SKIP LOCKED and a 60 s lease, so replicas never send the same callback concurrently. The POST carries the result JSON plus headers X-IL-Request-Id, X-Correlation-Id, X-IL-Attempt, X-IL-Timestamp and X-IL-Signature (sha256 HMAC of timestamp.body). |
| Retries  | Non-2xx or network error: retried after 10 s, 20 s, 40 s, ... (max 30 min); FAILED after 6 attempts. Every attempt is logged as an OUTBOUND row in api_call_log.                                                                                                                                  |
| Security | IL_CALLBACK_ALLOWED_HOSTS restricts callback_url hosts, which blocks server-side request forgery through the callback feature.                                                                                                                                                                    |

### 3.5 Scheduler (il-scheduler)

| Aspect | Detail                                                                                                                                                                      |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source | integration_layer/scheduler.py; same image as il-api, different command.                                                                                                    |
| Jobs   | reap_expired_leases every 60 s; ensure_month_partition for the current and next two months every 6 h; CALL dwh.load_incremental() every 15 min. Intervals are configurable. |

| Aspect          | Detail                                                                                                                                                                               |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Safety          | Each job takes a transaction-level advisory lock, so a second scheduler instance skips instead of duplicating work. Handles SIGTERM gracefully and reconnects after database errors. |
| Why not pg_cron | The official postgres image does not ship pg_cron; a container keeps the stack portable to any PostgreSQL, including managed services.                                               |

### 3.6 App2 (app2)

| Aspect            | Detail                                                                                                                                                                                                                        |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source            | app2/ : main.py (FastAPI), worker.py, queue_client.py, processors.py                                                                                                                                                          |
| Responsibility    | Consume requests for target_app = app2, run business logic, return output JSON and a workflow status.                                                                                                                         |
| Worker loop       | Up to 5 requests in parallel. Claims when slots are free; wakes on NOTIFY request_new, on task completion, or every 5 s. Heartbeat every lease/3 seconds.                                                                     |
| Business logic    | One async handler per workflow in processors.py (registered with @handler). Handlers raise RetryableError for temporary problems or PermanentError for unfixable ones; any other exception is treated as retryable.           |
| Output validation | Output is checked against output_schema before completing; a mismatch fails the request with OUTPUT_SCHEMA_INVALID.                                                                                                           |
| Database rights   | EXECUTE on claim_requests, heartbeat, complete_request, fail_request; SELECT on a few workflow_registry columns. No table access.                                                                                             |
| Endpoints         | /health, /health/ready, /stats (claimed, completed, retried, failed, lease_lost counters).                                                                                                                                    |
| Failure behaviour | Graceful shutdown finishes in-flight work (stop_grace_period 2 min). After a crash, leases expire and the reaper re-queues the requests. Handlers must be idempotent on request_id because a recovered request can run again. |
| Scaling           | Add replicas; SKIP LOCKED guarantees exclusive claims.                                                                                                                                                                        |

### 3.7 App1 mock and client library (app1-mock)

| Aspect         | Detail                                                                                                                                                            |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source         | app1_mock/main.py, app1_mock/il_client.py                                                                                                                         |
| Purpose        | Stand-in for the real App1 during development and testing. Replace it with the real application.                                                                  |
| Client library | IntegrationLayerClient: submit(), get(), cancel(), wait_for_result() for polling; verify_signature() for callbacks (checks HMAC and a 5-minute timestamp window). |
| Endpoints      | POST /orders (submit), GET /orders/{id} (poll), POST /hooks/il (callback receiver), GET /hooks/il/received.                                                       |

## 4. Integration Layer API

### 4.1 Endpoints

| Method | Path                                   | Caller | Purpose                                                                                          |
|--------|----------------------------------------|--------|--------------------------------------------------------------------------------------------------|
| POST   | /v1/requests                           | App    | Submit a request. 202 Accepted, or 200 for a duplicate idempotency_key                           |
| GET    | /v1/requests/{id}                      | App    | Status, output_json, errors, timestamps, callback state. ?include_input=true adds input_json     |
| GET    | /v1/requests                           | App    | List own requests; filters status, workflow_name, from_date, to_date; limit (max 200) and offset |
| POST   | /v1/requests/{id}/cancel               | App    | Cancel a PENDING request (409 otherwise)                                                         |
| GET    | /v1/workflows,<br>/v1/workflows/{name} | App    | Registered workflows and their JSON Schemas                                                      |

| Method | Path                           | Caller | Purpose                                                                              |
|--------|--------------------------------|--------|--------------------------------------------------------------------------------------|
| PUT    | /v1/admin/workflows/{name}     | Admin  | Create or update a workflow; a schema change bumps schema_version (201 when created) |
| POST   | /v1/admin/requests/{id}/replay | Admin  | Re-queue a FAILED request                                                            |
| GET    | /health, /health/ready         | Ops    | Liveness and database readiness                                                      |

## 4.2 Submit request body

```
{
  "workflow_name": "order_validation",           // must be registered and active
  "payload": { ...business JSON... },           // validated against input_schema
  "idempotency_key": "ORD-1001",                 // optional, max 200 chars
  "callback_url": "https://app1.internal/hook", // optional; host must be allow-listed
  "correlation_id": "uuid",                     // optional; generated if absent
  "priority": 0                                 // optional, 0-9 (higher first)
}
```

## 4.3 Error format and codes

```
{
  "error": {
    "code": "INPUT_SCHEMA_INVALID",
    "message": "payload does not match the workflow input schema",
    "schema_version": 1,
    "errors": [
      {
        "path": "$.items[0].quantity",
        "message": "0 is less than the minimum of 1",
        "rule": "minimum"
      }
    ]
  }
}
```

| HTTP | Code                             | When                                                           |
|------|----------------------------------|----------------------------------------------------------------|
| 401  | UNAUTHORIZED                     | Missing or invalid X-API-Key (or admin key)                    |
| 404  | NOT_FOUND                        | Unknown request id, or a request that belongs to another app   |
| 409  | NOT_CANCELLABLE / NOT_REPLAYABLE | Request is not in the required status                          |
| 413  | PAYLOAD_TOO_LARGE                | Body larger than IL_MAX_PAYLOAD_BYTES (1 MB)                   |
| 422  | REQUEST_INVALID                  | Envelope is malformed (missing or unknown fields, wrong types) |
| 422  | UNKNOWN_WORKFLOW                 | workflow_name is not registered or is inactive                 |
| 422  | INPUT_SCHEMA_INVALID             | payload fails the workflow's JSON Schema                       |
| 422  | CALLBACK_HOST_NOT_ALLOWED        | callback_url host is not in the allow-list                     |
| 422  | INVALID_JSON_SCHEMA              | Admin supplied a schema that is not valid JSON Schema          |

## 4.4 Callback payload

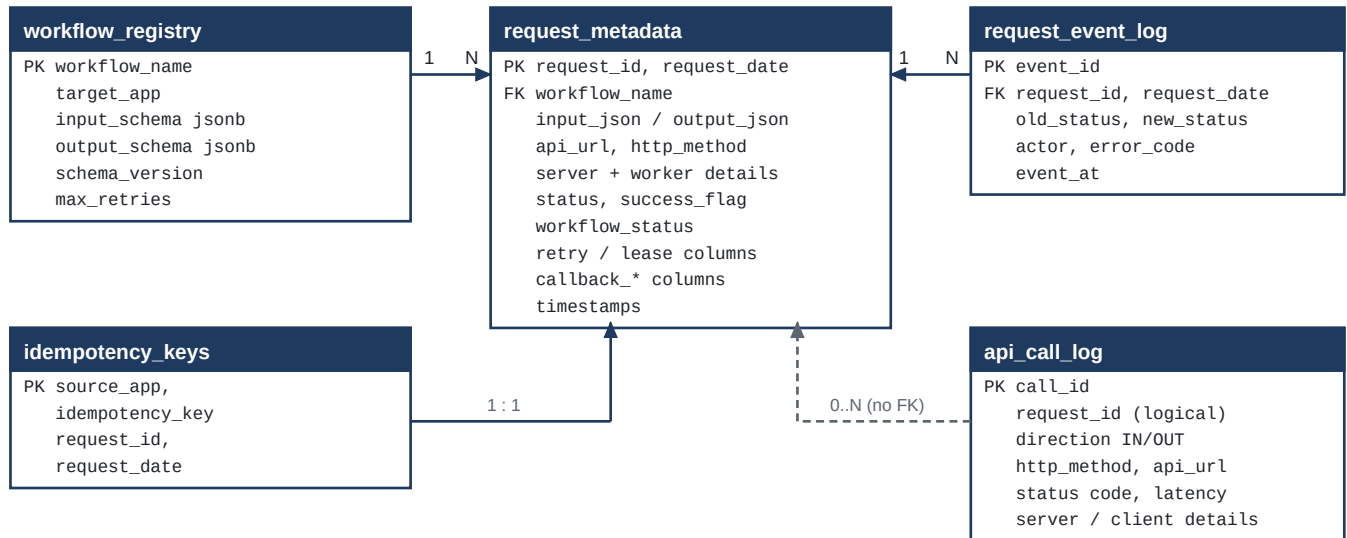
```
POST <callback_url>
X-IL-Request-Id, X-Correlation-Id, X-IL-Attempt, X-IL-Timestamp,
X-IL-Signature: sha256=HMAC_SHA256(secret, "<timestamp>.<raw body>")

{"event": "request.completed",           // request.failed | request.cancelled
 "request_id": "...", "correlation_id": "...", "workflow_name": "order_validation",
 "status": "COMPLETED", "success_flag": true, "workflow_status": "APPROVED",
 "output_json": {...}, "error": null,     // {"code", "message"} on failure
 "received_at": "...", "completed_at": "...", "attempt": 1}
```

App1 must answer with any 2xx status. Because a callback can be delivered more than once (for example if the response is lost), App1 should treat it idempotently by request\_id.

## 5. Database schema

The schema is defined in three migration files: `001_integration_schema.sql` (operational schema, functions, roles), `002_dwh_schema.sql` (warehouse) and `003_seed_workflows.sql` (example workflow). The column lists in this section were generated from the live database.



Solid = foreign key or unique mapping. Dashed = logical link (rejected calls have no `request_id`).

Figure 4. Operational schema (integration)

### 5.1 Requirement traceability: the metadata table

The original requirement asked for a metadata table storing the items below. All of them are columns of **integration.request\_metadata**, with detail tables for per-call and per-transition history.

| Requirement                    | Stored in                                                                                                                                                                                                      |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input JSON                     | <code>input_json</code> (jsonb, GIN-indexed) + <code>input_schema_version</code>                                                                                                                               |
| Date                           | <code>request_date</code> (date; also the partition key)                                                                                                                                                       |
| Timestamp                      | <code>received_at</code> , <code>picked_at</code> , <code>workflow_completed_at</code> , <code>callback_delivered_at</code> , <code>created_at</code> , <code>updated_at</code>                                |
| API URL, GET and POST requests | <code>api_url</code> + <code>http_method</code> for the submitting call; every GET/POST (inbound and outbound callbacks) in <code>api_call_log</code>                                                          |
| Server-level details           | <code>client_ip</code> , <code>il_server_hostname</code> , <code>il_server_ip</code> , <code>il_instance_id</code> , <code>app2_worker_host</code> , <code>app2_worker_id</code> , <code>db_server_addr</code> |
| Output JSON                    | <code>output_json</code> (jsonb)                                                                                                                                                                               |
| Workflow completion from App2  | <code>workflow_status</code> , <code>workflow_completed_at</code> , <code>processing_duration_ms</code>                                                                                                        |
| Success / failure flag         | <code>status</code> (enum) + <code>success_flag</code> (boolean) + <code>error_code</code> , <code>error_message</code>                                                                                        |

## 5.2 integration.request\_metadata

One row per request: the central metadata table. Range-partitioned by month on request\_date (request\_metadata\_YYYY\_MM plus a default partition). Primary key (request\_id, request\_date).

| Column                | Type           | Null | Description                                                 |
|-----------------------|----------------|------|-------------------------------------------------------------|
| request_id            | uuid           | no   | Primary key (with request_date); returned to App1           |
| request_date          | date           | no   | Submission date; monthly partition key                      |
| correlation_id        | uuid           | no   | End-to-end trace id (from App1 or generated)                |
| idempotency_key       | text           | yes  | App1-supplied key preventing duplicate submits              |
| source_app            | text           | no   | Calling application, derived from the API key               |
| target_app            | text           | no   | Consuming application, from workflow_registry               |
| workflow_name         | text           | no   | FK to workflow_registry                                     |
| priority              | smallint       | no   | 0-9, higher is claimed first                                |
| input_json            | jsonb          | no   | Validated request payload from App1                         |
| input_schema_version  | integer        | yes  | Registry schema version the payload was validated against   |
| output_json           | jsonb          | yes  | Result JSON from App2                                       |
| api_url               | text           | no   | Full URL of the submitting API call                         |
| http_method           | text           | no   | HTTP method of the submitting call                          |
| request_headers       | jsonb          | yes  | Request headers with secrets masked                         |
| callback_url          | text           | yes  | Where the result is POSTed (optional)                       |
| client_ip             | inet           | yes  | App1 caller IP                                              |
| il_server_hostname    | text           | yes  | IL host that accepted the request                           |
| il_server_ip          | inet           | yes  | IL host IP                                                  |
| il_instance_id        | text           | yes  | IL container / instance id                                  |
| app2_worker_host      | text           | yes  | Host of the App2 worker that processed it                   |
| app2_worker_id        | text           | yes  | App2 worker id (host-pid)                                   |
| db_server_addr        | inet           | yes  | Database server address                                     |
| status                | request_status | no   | PENDING, IN_PROGRESS, COMPLETED, FAILED, CANCELLED          |
| success_flag          | boolean        | yes  | TRUE completed, FALSE failed/cancelled, NULL in flight      |
| workflow_status       | text           | yes  | Workflow completion status reported by App2 (e.g. APPROVED) |
| error_code            | text           | yes  | Machine-readable error of the last failure                  |
| error_message         | text           | yes  | Human-readable error detail                                 |
| retry_count           | integer        | no   | Retries performed so far                                    |
| max_retries           | integer        | no   | Retry limit, copied from the registry                       |
| next_attempt_at       | timestampz     | no   | Earliest time the request may be claimed (backoff)          |
| locked_by             | text           | yes  | Worker currently holding the lease                          |
| locked_until          | timestampz     | yes  | Lease expiry; the reaper recovers expired leases            |
| received_at           | timestampz     | no   | Time the IL stored the request                              |
| picked_at             | timestampz     | yes  | Time App2 claimed it                                        |
| workflow_completed_at | timestampz     | yes  | Time App2 finished (success or final failure)               |

| Column                   | Type       | Null | Description                                      |
|--------------------------|------------|------|--------------------------------------------------|
| processing_duration_ms   | bigint     | yes  | picked_at to completion, in ms                   |
| callback_status          | text       | yes  | PENDING, DELIVERED, FAILED (NULL if no callback) |
| callback_attempts        | integer    | no   | Delivery attempts made                           |
| callback_next_attempt_at | timestampz | yes  | When the next delivery attempt is due            |
| callback_locked_until    | timestampz | yes  | Dispatcher lease for the current attempt         |
| callback_last_error      | text       | yes  | Error from the last failed attempt               |
| callback_delivered_at    | timestampz | yes  | Time App1 acknowledged the callback              |
| created_at               | timestampz | no   | Row creation time                                |
| updated_at               | timestampz | no   | Last change (trigger); DWH incremental watermark |

### 5.3 integration.workflow\_registry

Allowed workflows and their JSON Schemas. Seeded with `order_validation`; managed through PUT `/v1/admin/workflows/{name}`.

| Column         | Type       | Null | Description                                          |
|----------------|------------|------|------------------------------------------------------|
| workflow_name  | text       | no   | Primary key; lowercase identifier                    |
| target_app     | text       | no   | Application that processes this workflow             |
| description    | text       | yes  | Free-text description                                |
| input_schema   | jsonb      | no   | JSON Schema (2020-12) for App1's payload             |
| output_schema  | jsonb      | yes  | JSON Schema for App2's output (NULL = not validated) |
| schema_version | integer    | no   | Incremented whenever a schema changes                |
| max_retries    | integer    | no   | Retry limit for new requests (0-20)                  |
| is_active      | boolean    | no   | Inactive workflows reject new submissions            |
| created_at     | timestampz | no   | Creation time                                        |
| updated_at     | timestampz | no   | Last change (trigger)                                |

### 5.4 integration.api\_call\_log

Every HTTP call handled or made by the Integration Layer, including rejected ones (401, 422) and outbound callbacks.

| Column             | Type    | Null | Description                                                 |
|--------------------|---------|------|-------------------------------------------------------------|
| call_id            | bigint  | no   | Primary key (identity)                                      |
| request_id         | uuid    | yes  | Related request (NULL when rejected before storing)         |
| correlation_id     | uuid    | yes  | Trace id                                                    |
| direction          | text    | no   | INBOUND (App1 to IL) or OUTBOUND (callback)                 |
| http_method        | text    | no   | GET, POST, ...                                              |
| api_url            | text    | no   | Full URL                                                    |
| endpoint           | text    | yes  | Route template, e.g. <code>/v1/requests/{request_id}</code> |
| query_params       | jsonb   | yes  | Query string as JSON                                        |
| request_headers    | jsonb   | yes  | Headers, secrets masked                                     |
| request_body_bytes | integer | yes  | Request body size                                           |

| Column               | Type       | Null | Description                                   |
|----------------------|------------|------|-----------------------------------------------|
| response_status_code | integer    | yes  | HTTP status returned or received              |
| response_body        | jsonb      | yes  | Error body (successful bodies are not stored) |
| latency_ms           | integer    | yes  | Duration in ms                                |
| client_ip            | inet       | yes  | Caller IP                                     |
| user_agent           | text       | yes  | Caller user agent                             |
| server_hostname      | text       | yes  | IL host                                       |
| server_ip            | inet       | yes  | IL host IP                                    |
| instance_id          | text       | yes  | IL instance                                   |
| called_at            | timestampz | no   | Time of the call (BRIN index)                 |

## 5.5 integration.request\_event\_log

Audit trail written by a trigger on every insert and status change; the source for SLA analysis.

| Column        | Type           | Null | Description                                |
|---------------|----------------|------|--------------------------------------------|
| event_id      | bigint         | no   | Primary key (identity)                     |
| request_id    | uuid           | no   | FK (with request_date) to request_metadata |
| request_date  | date           | no   | Partition key of the parent row            |
| old_status    | request_status | yes  | Status before (NULL on insert)             |
| new_status    | request_status | no   | Status after                               |
| retry_count   | integer        | yes  | Retry count at the time                    |
| actor         | text           | yes  | il, worker id, reaper or admin-replay      |
| error_code    | text           | yes  | Error at the time                          |
| error_message | text           | yes  | Error detail                               |
| event_at      | timestampz     | no   | Transition time                            |

## 5.6 integration.idempotency\_keys

Global uniqueness of (source\_app, idempotency\_key). It is a separate table because a partitioned table cannot enforce uniqueness on columns that exclude the partition key.

| Column          | Type       | Null | Description           |
|-----------------|------------|------|-----------------------|
| source_app      | text       | no   | PK part: calling app  |
| idempotency_key | text       | no   | PK part: App1's key   |
| request_id      | uuid       | no   | Original request      |
| request_date    | date       | no   | Original request date |
| created_at      | timestampz | no   | First seen            |

## 5.7 Functions and procedures

| Function                     | Granted to | Behaviour                                                                                               |
|------------------------------|------------|---------------------------------------------------------------------------------------------------------|
| load_incremental (procedure) | dwh_etl    | Procedure: upserts dimensions and fact_request from rows changed since the watermark (5-minute overlap) |
| cancel_request               | il_service | PENDING to CANCELLED                                                                                    |

| Function               | Granted to   | Behaviour                                                                                                              |
|------------------------|--------------|------------------------------------------------------------------------------------------------------------------------|
| claim_callbacks        | il_service   | Claims due callbacks with SKIP LOCKED and returns the payload fields                                                   |
| claim_requests         | app2_worker  | Claims up to N due PENDING rows with SKIP LOCKED, sets IN_PROGRESS and the lease                                       |
| complete_request       | app2_worker  | Stores output_json and workflow_status; COMPLETED. Returns false if the lease was lost                                 |
| ensure_month_partition | il_scheduler | Creates the monthly partition if missing                                                                               |
| fail_request           | app2_worker  | Retryable and retries left: back to PENDING with backoff; otherwise FAILED                                             |
| heartbeat              | app2_worker  | Extends the lease while the holder is still processing                                                                 |
| reap_expired_leases    | il_scheduler | Re-queues (or fails) IN_PROGRESS rows whose lease expired                                                              |
| record_callback_result | il_service   | DELIVERED, or retry with backoff, or FAILED after max attempts                                                         |
| replay_request         | il_service   | FAILED back to PENDING with retries reset (admin)                                                                      |
| submit_request         | il_service   | Checks the workflow is active, applies idempotency, inserts the PENDING row with metadata; returns id and is_duplicate |
| trg_schedule_callback  | trigger      | BEFORE UPDATE OF status: queues callback on terminal status                                                            |
| trg_set_updated_at     | trigger      | BEFORE UPDATE: maintains updated_at                                                                                    |
| trg_status_change      | trigger      | AFTER INSERT/UPDATE OF status: audit row + NOTIFY                                                                      |

All non-trigger functions are SECURITY DEFINER with a fixed search\_path, and EXECUTE is revoked from PUBLIC, so each role can call only the functions granted to it.

## 5.8 Indexes

| Table           | Index                               | Definition                     |
|-----------------|-------------------------------------|--------------------------------|
| api_call_log    | api_call_log_pkey                   | btree (call_id)                |
| api_call_log    | ix_acl_called_at                    | brin (called_at)               |
| api_call_log    | ix_acl_request_id                   | btree (request_id)             |
| dim_application | dim_application_app_code_key        | btree (app_code)               |
| dim_application | dim_application_pkey                | btree (app_key)                |
| dim_date        | dim_date_full_date_key              | btree (full_date)              |
| dim_date        | dim_date_pkey                       | btree (date_key)               |
| dim_server      | dim_server_pkey                     | btree (server_key)             |
| dim_server      | dim_server_server_role_hostname_key | btree (server_role, hostname)  |
| dim_status      | dim_status_pkey                     | btree (status_key)             |
| dim_status      | dim_status_status_code_key          | btree (status_code)            |
| dim_workflow    | dim_workflow_pkey                   | btree (workflow_key)           |
| dim_workflow    | dim_workflow_workflow_name_key      | btree (workflow_name)          |
| etl_watermark   | etl_watermark_pkey                  | btree (job_name)               |
| fact_request    | fact_request_pkey                   | btree (request_id)             |
| fact_request    | ix_fr_date                          | btree (date_key)               |
| fact_request    | ix_fr_status                        | btree (status_key)             |
| fact_request    | ix_fr_workflow                      | btree (workflow_key, date_key) |

| Table             | Index                  | Definition                                                                                                |
|-------------------|------------------------|-----------------------------------------------------------------------------------------------------------|
| idempotency_keys  | idempotency_keys_pkey  | btree (source_app, idempotency_key)                                                                       |
| request_event_log | ix_rel_request         | btree (request_id, event_at)                                                                              |
| request_event_log | request_event_log_pkey | btree (event_id)                                                                                          |
| request_metadata  | ix_rm_callback_due     | btree (callback_next_attempt_at) WHERE (callback_status = 'PENDING'::text)                                |
| request_metadata  | ix_rm_correlation_id   | btree (correlation_id)                                                                                    |
| request_metadata  | ix_rm_inprogress_lease | btree (locked_until) WHERE (status = 'IN_PROGRESS'::integration.request_status)                           |
| request_metadata  | ix_rm_input_gin        | gin (input_json jsonb_path_ops)                                                                           |
| request_metadata  | ix_rm_pending          | btree (target_app, priority DESC, next_attempt_at) WHERE (status = 'PENDING'::integration.request_status) |
| request_metadata  | ix_rm_request_id       | btree (request_id)                                                                                        |
| request_metadata  | ix_rm_status_date      | btree (status, request_date)                                                                              |
| request_metadata  | ix_rm_updated_at       | btree (updated_at)                                                                                        |
| request_metadata  | request_metadata_pkey  | btree (request_id, request_date)                                                                          |
| workflow_registry | workflow_registry_pkey | btree (workflow_name)                                                                                     |

The partial indexes (ix\_rm\_pending, ix\_rm\_inprogress\_lease, ix\_rm\_callback\_due) keep the queue queries fast regardless of how much history accumulates, because they only contain in-flight rows.

## 5.9 Roles and privileges

| Group role   | Login user              | Privileges                                                                                                                                                                   |
|--------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| il_service   | il_api                  | EXECUTE submit, cancel, replay, claim_callbacks, record_callback_result; SELECT v_request_status; SELECT/INSERT/UPDATE workflow_registry; INSERT/SELECT api_call_log         |
| app2_worker  | app2                    | EXECUTE claim_requests, heartbeat, complete_request, fail_request; SELECT on workflow_registry columns (workflow_name, target_app, output_schema, schema_version, is_active) |
| il_scheduler | il_scheduler_user       | EXECUTE reap_expired_leases, ensure_month_partition; member of dwh_etl                                                                                                       |
| dwh_etl      | (via il_scheduler_user) | SELECT on all integration tables; read/write dwh; EXECUTE load_incremental                                                                                                   |
| dwh_reader   | bi_reader               | SELECT on dwh only                                                                                                                                                           |

**Verified:** the app2 and bi\_reader users are refused direct reads of request\_metadata (InsufficientPrivilege), and the migrator is idempotent.

## 6. Data warehouse layer

The **dwh** schema is a star model with one fact row per request. It is loaded every 15 minutes by the scheduler calling `dwh.load_incremental()`, and read by BI tools through the `bi_reader` user.

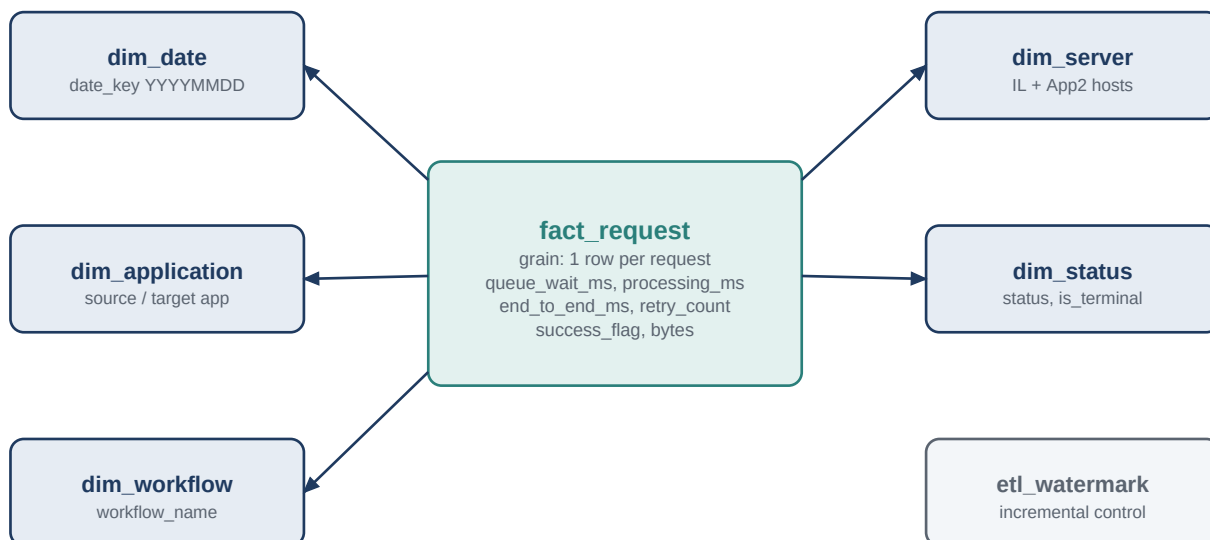


Figure 5. Warehouse star schema

### 6.1 Load process

- Read the watermark from `dwh.etl_watermark` and select rows from `request_metadata` with `updated_at` later than the watermark minus 5 minutes. The overlap catches transactions that committed late.
- Upsert new applications, workflows and servers into the dimensions (unknown servers map to an UNKNOWN member).
- Upsert `fact_request` by `request_id`, updating only when the source row is newer. Re-running the load is always safe; a request's fact row keeps changing until it reaches a final status.
- Advance the watermark to the latest `updated_at` processed and record the row count.

### 6.2 fact\_request

| Column                         | Type     | Null | Description                                 |
|--------------------------------|----------|------|---------------------------------------------|
| <code>request_id</code>        | uuid     | no   | Primary key; same id as the source request  |
| <code>correlation_id</code>    | uuid     | no   | Trace id                                    |
| <code>date_key</code>          | integer  | no   | FK <code>dim_date</code> (request date)     |
| <code>source_app_key</code>    | integer  | no   | FK <code>dim_application</code> (caller)    |
| <code>target_app_key</code>    | integer  | no   | FK <code>dim_application</code> (consumer)  |
| <code>workflow_key</code>      | integer  | no   | FK <code>dim_workflow</code>                |
| <code>il_server_key</code>     | integer  | no   | FK <code>dim_server</code> (IL host)        |
| <code>worker_server_key</code> | integer  | no   | FK <code>dim_server</code> (App2 host)      |
| <code>status_key</code>        | smallint | no   | FK <code>dim_status</code> (current status) |
| <code>success_flag</code>      | boolean  | yes  | Final outcome (NULL while in flight)        |
| <code>workflow_status</code>   | text     | yes  | App2's completion status                    |

| Column            | Type       | Null | Description                           |
|-------------------|------------|------|---------------------------------------|
| error_code        | text       | yes  | Last error code                       |
| retry_count       | integer    | no   | Retries used                          |
| api_call_count    | integer    | no   | Rows in api_call_log for this request |
| queue_wait_ms     | bigint     | yes  | received_at to picked_at              |
| processing_ms     | bigint     | yes  | picked_at to completion               |
| end_to_end_ms     | bigint     | yes  | received_at to completion             |
| input_bytes       | integer    | yes  | Size of input_json                    |
| output_bytes      | integer    | yes  | Size of output_json                   |
| received_at       | timestampz | no   | Submission time                       |
| completed_at      | timestampz | yes  | Completion time                       |
| source_updated_at | timestampz | no   | Source row version used for upserts   |
| etl_loaded_at     | timestampz | no   | Time of the last load of this row     |

## 6.3 Dimensions and control tables

| Table               | Columns                                                                                                          | Notes                                               |
|---------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| dwh.dim_date        | date_key, full_date, year, quarter, month, month_name, day_of_month, day_of_week, day_name, iso_week, is_weekend | Pre-populated 2024-2035; date_key YYYYMMDD          |
| dwh.dim_application | app_key, app_code                                                                                                | source and target apps                              |
| dwh.dim_workflow    | workflow_key, workflow_name                                                                                      | One row per workflow_name                           |
| dwh.dim_server      | server_key, server_role, hostname                                                                                | server_role IL / APP2_WORKER / UNKNOWN              |
| dwh.dim_status      | status_key, status_code, is_terminal, is_success                                                                 | Seeded with the 5 statuses, is_terminal, is_success |
| dwh.etl_watermark   | job_name, last_loaded_at, last_run_at, rows_loaded                                                               | Incremental load control                            |

## 6.4 Reporting view

dwh.v\_daily\_workflow\_summary gives, per day and workflow: total requests, completed, failed, in flight, success rate %, p50 and p95 end-to-end latency in ms, and total retries.

```
SELECT * FROM dwh.v_daily_workflow_summary ORDER BY full_date DESC;
```

|            |                  |       |           |        |           |                  |
|------------|------------------|-------|-----------|--------|-----------|------------------|
| full_date  | workflow_name    | total | completed | failed | in_flight | success_rate_pct |
| 2026-09-28 | order_validation | 3     | 2         | 0      | 1         | 100.00           |

## 7. Security, reliability and operations

### 7.1 Security controls

| Area              | Control                                                                                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication    | Per-application API keys (constant-time comparison); separate admin key                                                                                    |
| Authorization     | Apps see and cancel only their own requests; admin endpoints separated                                                                                     |
| Database          | Least-privilege roles; App1 has no DB access; App2 limited to four functions; SECURITY DEFINER functions with fixed search_path; PUBLIC privileges revoked |
| Input handling    | Strict envelope (unknown fields rejected), JSON Schema validation, 1 MB size limit, invalid client IPs ignored                                             |
| Secrets in logs   | Authorization, X-API-Key, Cookie and Set-Cookie headers masked before storage                                                                              |
| Callbacks         | HMAC-SHA256 signature with timestamp (replay window 5 min); host allow-list against SSRF                                                                   |
| Network           | Postgres published on 127.0.0.1 only; containers run as a non-root user                                                                                    |
| Pending (Phase 6) | TLS termination proxy, secrets from a vault or Docker secrets instead of .env                                                                              |

### 7.2 Reliability mechanisms

| Risk                                 | Mitigation                                                                                                   |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Two workers process the same request | FOR UPDATE SKIP LOCKED claims; only the lease holder can complete or fail (tested with 4 concurrent workers) |
| App2 crashes mid-job                 | Lease expiry + reaper re-queues within about a minute; retry_count increases                                 |
| Temporary downstream failure         | Retryable failure with exponential backoff up to max_retries                                                 |
| App1 resubmits after a timeout       | idempotency_key returns the original request                                                                 |
| Missed NOTIFY (reconnect, restart)   | Every listener also polls (App2 5 s, dispatcher 5 s)                                                         |
| App1 callback endpoint down          | Callback retries with backoff, 6 attempts; polling still works                                               |
| Partition missing for a new month    | Scheduler pre-creates 3 months; a DEFAULT partition catches anything else                                    |
| Schema drift between environments    | Checksummed migrations; changed files are refused                                                            |

### 7.3 Configuration reference

| Variable                   | Service      | Default / purpose                  |
|----------------------------|--------------|------------------------------------|
| IL_DATABASE_URL            | il-api       | Connection for il_api user         |
| IL_API_KEYS                | il-api       | app1:<key>[,app3:<key>]            |
| IL_ADMIN_API_KEY           | il-api       | Key for /v1/admin/*                |
| IL_CALLBACK_SIGNING_SECRET | il-api       | HMAC secret shared with App1       |
| IL_CALLBACK_ALLOWED_HOSTS  | il-api       | Comma-separated hosts; empty = any |
| IL_CALLBACK_MAX_ATTEMPTS   | il-api       | 6                                  |
| IL_MAX_PAYLOAD_BYTES       | il-api       | 1000000                            |
| IL_SCHEMA_CACHE_SECONDS    | il-api       | 30                                 |
| APP2_DATABASE_URL          | app2         | Connection for app2 user           |
| APP2_MAX_CONCURRENCY       | app2         | 5 requests in parallel             |
| APP2_LEASE_SECONDS         | app2         | 120 (heartbeat every 40 s)         |
| SCHEDULER_DATABASE_URL     | il-scheduler | Connection for il_scheduler_user   |

| Variable                                 | Service      | Default / purpose                             |
|------------------------------------------|--------------|-----------------------------------------------|
| SCHEDULER_*_INTERVAL_SECONDS             | il-scheduler | reap 60, partition 21600, dwh 900             |
| MIGRATE_DATABASE_URL, MIGRATE_*_PASSWORD | migrate      | Superuser connection and login-user passwords |

## 8. Deployment

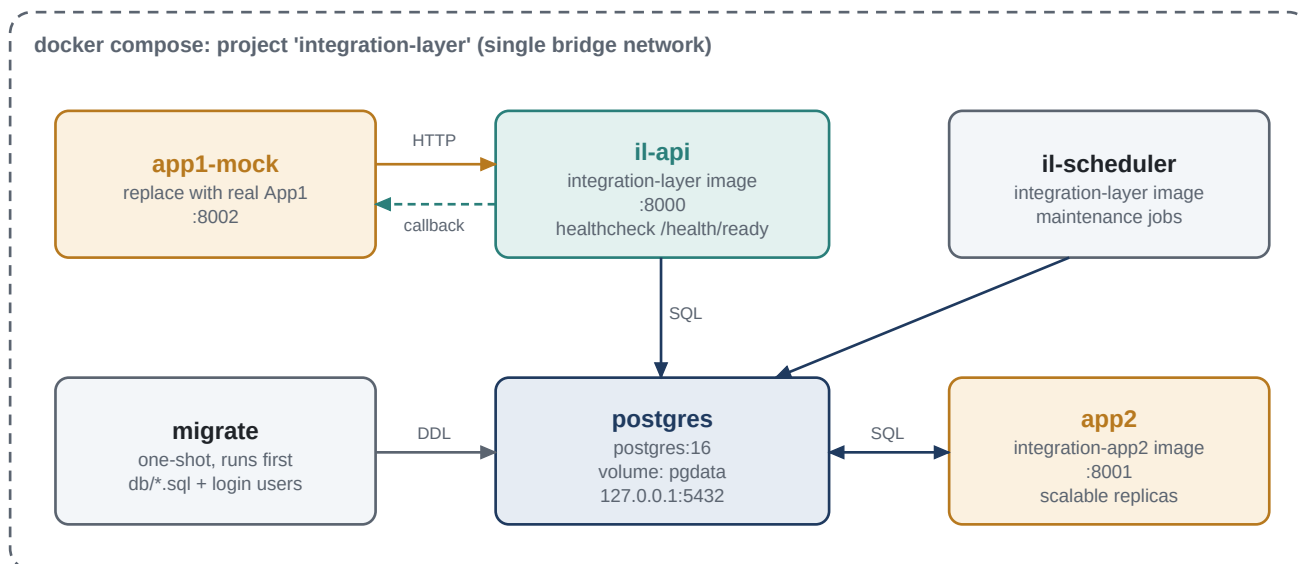


Figure 6. Docker Compose deployment

Start-up order is enforced by Compose: postgres (healthy) then migrate (exits 0) then il-api, il-scheduler and app2; app1-mock waits for il-api to be healthy. il-api and app2 have health checks against /health/ready.

```

cp .env.example .env          # set every password and secret
docker compose up -d --build
docker compose ps             # migrate: exited (0); others: healthy/running
python scripts/smoke_test.py  # end-to-end verification
  
```

### 8.1 Adding a new workflow

- Register it: PUT /v1/admin/workflows/{name} with target\_app, input\_schema, output\_schema and max\_retries.
- Implement an async handler in app2/processors.py decorated with @handler("{name}") that returns Result(output, workflow\_status).
- Deploy App2. Requests for a workflow without a handler fail with NO\_HANDLER, so deploy the handler before App1 starts sending.

## 9. Verification

Tests ran against PostgreSQL 16.4 with the three services running locally under uvicorn.

| Suite                         | Result    | Coverage                                                                                                                                                                                                     |
|-------------------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pytest: database              | 13 passed | Exclusive claims with 4 concurrent workers, idempotency, completion, lease-holder check, retries and backoff, permanent failure, reaper, callback queue/backoff/give-up, audit trail, roles, DWH idempotency |
| pytest: Integration Layer API | 13 passed | Auth, submit and poll, ownership, schema errors by JSON path, envelope errors, callback allow-list, duplicates, cancel, listing, replay, workflow versioning, API log masking, forged X-Forwarded-For        |

| Suite                 | Result         | Coverage                                                                                                                         |
|-----------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------|
| pytest: App2 worker   | 3 passed       | Business rules, callback signature, all six worker outcomes (completed, invalid output, retryable, permanent, crash, no handler) |
| End-to-end smoke test | 21 / 21 passed | App1 to IL to PostgreSQL to App2 to signed callback; polling path; business rejection; retryable failure; duplicates             |

**Not yet executed:** `docker compose up`. The Docker daemon was not running on the build machine. The Compose file passes `docker compose config` and every pinned dependency has Python 3.12 Linux wheels for x86\_64 and arm64.

## 10. Status and roadmap

| Phase | Scope                                                     | Status                             |
|-------|-----------------------------------------------------------|------------------------------------|
| 1     | Database schema, functions, roles, DWH                    | Done, tested                       |
| 1b    | Scheduled jobs (reaper, partitions, DWH load)             | Done                               |
| 2     | Integration Layer API, validation, API logging, callbacks | Done, tested                       |
| 3     | App2 worker service                                       | Done, tested                       |
| 4     | Docker Compose, App1 mock, smoke test                     | Built; Docker run pending          |
| 5     | Reporting: BI tool connection, extra marts                | Schema, load and summary view done |
| 6     | Hardening and operations                                  | Not started                        |

### 10.1 Next steps

- Run the stack with Docker on the target host and execute the smoke test.
- Replace app1-mock with the real App1, using `il_client.py` or plain HTTP.
- Implement the real workflows in `app2/processors.py` and register their JSON Schemas.
- Connect a BI tool (for example Metabase) with the `bi_reader` user.
- Hardening: TLS proxy in front of `il-api`, secrets management, nightly backups (`pg_dump` or WAL archiving), 90-day partition retention, Prometheus metrics and alerts (oldest PENDING, failure rate, callback failures), structured JSON logs, CI pipeline.